

리눅스, fork에서 io_uring까지, OS가 선택한 타협

갑자기 서버에 접속이 안 됩니다. 분명히 SSH 키를 넣었는데 `Permission denied (publickey)`.
줄면서 vim을 사용하다보니 `authorized_keys` 를 실수로 덮어썼던 겁니다. 이러한 경험은 아마 한
번쯤 겪어보셨을 겁니다. 그 경...

A

2026년 3월 1일 16분 읽기 Admin

#linux #os #security #ssh #teleport #selinux #devops #kernel

갑자기 서버에 접속이 안 됩니다.

분명히 SSH 키를 넣었는데 `Permission denied (publickey)`. 줄면서 vim을 사용하다보니
`authorized_keys` 를 실수로 덮어썼던 겁니다. 이러한 경험은 아마 한 번쯤 겪어보셨을 겁니다.

그 경험 이후로 저는 생각했습니다. "우리는 왜 *열쇠 뭉치*를 *손에 쥐고* 서버를 관리하는가?" 자물쇠가 수백 개라면,
열쇠도 수백 개여야 할까? 아니면 문을 여는 방식 자체를 다시 설계해야 할까?

이 글은 그 질문에서 출발합니다. SSH 보안의 패러다임을 넘어, 리눅스가 수십 년에 걸쳐 정제해 온 세 가지 설계를
함께 읽어보려 합니다.

기술의 역사는 언제나 불편함에서 시작되며, 참신한 추상으로 끝났기 때문입니다.

1부. 보안의 계층화 — 열쇠에서 신뢰로

1. 왜 우리는 '잠금'에 집착하게 되었는가

전통적인 리눅스 보안은 **DAC(임의 접근 제어, Discretionary Access Control)** 모델에 기반합니다. 파일
소유자가 권한을 '임의로' 결정할 수 있는 이 방식은 유연하지만, 대규모 환경에서는 치명적인 약점을 노출합니다.

파일 권한을 설정하고 SSH 키를 배포하는 일이 마치 '열쇠 뭉치를 배부하는 것'과 같다는 걸, 시스템이 커질수록
실감하게 됩니다.

- 1. Private Key의 안티 패턴:** 개인키를 중앙 서버에 모아두는 것은 "모든 문을 열 수 있는 마스터 키를 한 곳에 두는 것"과 같습니다. 중앙 서버 탈취는 곧 전체 인프라의 붕괴를 의미하죠.
- 2. 권한의 과잉(Over-privilege):** 특정 프로세스가 해킹당하면 해당 프로세스를 실행한 사용자의 권한을 그대로 상속받아 시스템 전체가 위협받습니다.
- 3. 최소 권한 원칙의 부재:** 루트(root) 사용자는 모든 보안 정책을 우회할 수 있어, 단일 실패 지점(Single Point of Failure)이 됩니다.

이러한 한계를 극복하기 위해 **MAC(강제 접근 제어, Mandatory Access Control)** 와 **SELinux** 가 탄생했습니다. 이는 사용자 개인이 아닌 '시스템 정책'이 접근 여부를 최종 결정하며, 설령 루트 권한을 얻더라도 정의된 동작(Sandbox) 외에는 아무것도 하지 못하게 격리하는 철학을 담고 있습니다.

열쇠를 관리하는 것이 아니라, 문을 여는 '자격'을 관리하는 시대로의 전환입니다.

2. 현대적 보안 인프라의 거시적 풍경

현대적인 보안 인프라는 단일 도구가 아닌, 여러 계층이 유기적으로 맞물린 **Defense in Depth(심층 방어)** 구조를 가집니다. 마치 오케스트라처럼, 각 파트가 독립적으로 연주하면서도 하나의 화음을 만들어내는 것과 비슷합니다.

- 1. Unified Access Gateway (Teleport):** 가장 상위 계층에서 기존의 영구적인 SSH 키를 폐기하고, IDP(Okta, Google 등)와 연동된 **단기 인증서(Short-lived Certificates)** 방식을 채택합니다. 모든 세션은 녹화되며 게이트웨이 단에서 RBAC가 통합 제어됩니다. 1주일 뒤 만료되는 방문증을 발급하는 임시 안내데스크를 상상해 보세요. 영구 키 분실의 공포가 사라집니다.
- 2. 인증 프레임워크 (PAM & SSSD):** 시스템 입구에서 **PAM(Pluggable Authentication Modules)** 이 MFA 적용, 접속 시간 제한 등을 수행합니다. **SSSD** 는 중앙 집중식 인증 서버(FreeIPA, AD)와 로컬 시스템을 안전하게 연결합니다.
- 3. 커널 레벨 통제 (SELinux):** 인증을 통과한 후에도 모든 행위는 커널의 LSM(Linux Security Module)을 통해 감시됩니다. 프로세스 도메인과 파일 컨텍스트를 대조하여 허가되지 않은 시스템 콜을 원천 차단합니다.

3. 블랙박스를 걷어내며

보안은 투명해야 합니다. 믿음이 아니라 증거로 확인되어야 비로소 신뢰를 가질 수 있습니다. GUI 뒤에 숨겨진 정책의 흐름을 터미널에서 직접 읽어보는 경험은, 시스템을 바라보는 시선 자체를 다르게 가질 수 있습니다.

¹ # 프로세스가 현재 어떤 '도메인'에서 실행 중인지 확인합니다

```
2 ps -eZ | grep httpd
3 # → system_u:system_r:httpd_t:s0
4 # 이 문맥이 곧 httpd가 할 수 있는 일과 할 수 없는 일의 경계입니다
```

```
1 # ls -l에서는 보이지 않는 세밀한 권한을 투명하게 읽어냅니다
2 getfacl /data/shared_project
3
4 # SELinux가 차단한 행위의 근거를 추적합니다
5 ausearch -m avc -ts recent
```

`ausearch`의 출력 한 줄이 "왜 서비스가 갑자기 동작하지 않지?"라는 질문에 답을 줍니다.

4. 설계 구현방법

원리와 구조를 이해한 뒤에는, 실제로 손을 움직일 차례입니다.

소유권을 건드리지 않고 특정 사용자에게만 권한을 부여하려면:

```
setfacl -m u:developer:rw /var/log/deploy.log
```

파일이 이동하면서 잃어버린 SELinux 보안 라벨을 복구하려면:

```
1 restorecon -vR /var/www/html/
2 # 이 한 줄이 "서비스가 갑자기 503을 반환하는" 미스터리를 해결하는 경우가 적지 않습니다
```

표준 포트가 아닌 환경에서 서비스를 허용하려면:

```
semanage port -a -t http_port_t -p tcp 8081
```

보안 1부를 마치며 — 열쇠에서 신뢰로

SSH 키 관리의 스트레스에서 벗어나는 진정한 방법은 더 좋은 관리 도구를 찾는 것이 아닙니다. **보안의 계층화와 강제 접근 제어라는 패러다임에 적응하는 것이죠.** Teleport로 입구를 단일화하고, SELinux로 내부를 격리하며, ACL로 세밀하게 조정하는 이 구조는 "열쇠를 잘 관리하자"가 아니라 "열쇠라는 개념 자체를 다시 정의하자"는 변화입니다.

2부. fork()와 exec() — 분리가 만들어 낸 자유

1. 왜 UNIX는 프로세스 탄생을 두 단계로 나눴는가

터미널에서 `ls | grep ".txt" | wc -l` 을 입력하는 순간, 우리는 세 개의 독립적인 프로그램이 한 줄의 데이터 파이프라인으로 연결되는 마법을 경험합니다. 이것이 가능한 이유는 UNIX가 1969년에 내린 하나의 설계 결정 덕분입니다. **프로세스를 '복제'하는 것과 '새 프로그램을 실행'하는 것을 분리한다.**

당시 다른 OS들(VMS, OS/360)은 `spawn()` 이나 `CreateProcess()` 처럼 프로세스 생성과 프로그램 실행을 **하나의 원자적 연산**으로 묶었습니다. 직관적이지만 치명적인 제약이 있었죠. 새 프로세스가 시작되기 **전에** 환경을 세밀하게 조정할 창구가 없었습니다.

UNIX는 달랐습니다.

- `fork()` 는 부모 프로세스를 복제해 자식을 만듭니다. 자식은 부모의 파일 디스크립터, 메모리, 환경 변수를 고스란히 상속받습니다.
- `exec()` 는 그 자식의 메모리 이미지를 새로운 실행 파일로 덮어씹습니다. PID는 그대로지만, 코드와 데이터와 스택은 완전히 교체됩니다.

그리고 그 두 단계 사이에 **황금 틈(golden window)** 이 생깁니다. 이 틈에서 자식 프로세스는 `exec()` 를 호출하기 전에 자신의 입출력 환경을 마음껏 재설계할 수 있습니다. 쉘 파이프라인, 리다이렉션, 백그라운드 실행 — 이 모든 것은 그 틈에서 태어났습니다.

"하나의 일을 잘 하는 작은 프로그램들을 조합하라." — Doug McIlroy

fork/exec의 분리는 이 철학을 OS 레벨에서 구현한 기반 메커니즘입니다.

2. 프로세스 탄생의 구조 — 커널이 관리하는 생명 목록

커널은 모든 프로세스를 `task_struct` 라는 자료구조로 관리합니다. `fork()` 호출 순간, 커널이 하는 일을 조감도처럼 바라봐 봅시다.

[부모 <code>task_struct</code>]	[자식 <code>task_struct</code>] — <code>fork</code> 직후
pid: 1001	pid: 1002 (새로 할당)
ppid: 1000	ppid: 1001 (부모의 pid)
fd_table → [0, 1, 2, ...]	fd_table → 동일한 FD (참조 카운트만 증가)
mm_struct → [페이지 테이블]	mm_struct → Copy-on-Write 페이지 테이블
signal_handlers	signal_handlers (상속)

Copy-on-Write(CoW) 는 이 과정의 숨은 영웅입니다. `fork()` 직후 부모와 자식은 동일한 물리 메모리 페이지를 공유합니다. 어느 쪽이 페이지를 수정하려는 순간, 페이지 폴트가 발생하고 커널이 그때서야 복사합니다. `exec()` 를 즉시 호출하는 평범한 패턴에서는 CoW 덕분에 메모리 복사 비용이 거의 0에 가깝습니다.

파이프라인 `ls -l | grep ".txt" | wc -l` 에서 셸 내부가 하는 일을 도식으로 보면 황금 튜의 의미가 명확해집니다.

```
Shell (pid: 100)
|
├─ fork() → Child1 (pid: 101) [황금 튜]
|   pipe() → pipefd[0](읽기), pipefd[1](쓰기)
|   dup2(pipefd[1], STDOUT_FILENO) ← stdout을 파이프 쓰기로 교체
|   exec("ls", "-l")                ← 이제 새 프로그램으로 교체
|
├─ fork() → Child2 (pid: 102) [황금 튜]
|   dup2(pipefd[0], STDIN_FILENO) ← stdin을 파이프 읽기로 교체
|   exec("grep", ".txt")
|
└─ fork() → Child3 (pid: 103)
    exec("wc", "-l")
```

`exec()` 계열 함수는 이름이 여섯 가지나 되지만, 모두 내부적으로 단 하나의 시스템 콜 `execve()` 로 수렴합니다. 나머지는 glibc가 제공하는 편의 래퍼입니다.

함수	인자 방식	환경 변수	PATH 탐색
<code>execl</code>	가변 인자(리스트)	상속	X
<code>execv</code>	배열 포인터	상속	X
<code>execle</code>	가변 인자	명시 지정	X
<code>execve</code>	배열 포인터	명시 지정	X
<code>execlp</code>	가변 인자	상속	✓
<code>execvp</code>	배열 포인터	상속	✓

3. 투명하게 들여다보기 — strace와 /proc로 제어 흐름 추적

"정말 그렇게 동작하나요?" 가장 좋은 답은 직접 확인입니다.

```

1 # 셸이 "ls"를 실행할 때 fork/exec가 어떻게 호출되는지 실시간 추적
2 strace -e trace=clone,execve,wait4 -f bash -c "ls /tmp"

```

출력은 이렇습니다.

```

clone(...) = 12345          # fork() ≈ clone()
[pid 12345] execve("/usr/bin/ls", ...) = 0 # 새 프로그램으로 교체
[pid 9000] wait4(12345, ...)   # 부모 셸이 자식을 기다림

```

숫자가 아니라 이야기가 보이시나요? 부모(셸)가 자식을 낳고, 자식이 새 존재로 탈바꿈하고, 부모가 그 결과를 기다리는 구조입니다.

```
1 # 백그라운드(&)는 이 wait4가 없는 것
2 strace -e trace=wait4 bash -c "sleep 2 & echo 'shell continues'"
3 # → wait4가 호출되지 않음을 확인. 부모가 자식을 기다리지 않습니다.
4
5 # /proc으로 프로세스 계층 탐색
6 cat /proc/$$/status | grep -E "Pid|PPid"
7 ls -la /proc/$$/fd # 현재 셸이 열고 있는 파일 디스크립터 목록
```

4. 직접 써보기

이해는 코드를 직접 써볼 때 비로소 체화됩니다. `ls -l | grep .c` 를 C로 구현해봅시다.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pipe(pipefd); // 파이프 생성: [0]=읽기, [1]=쓰기

    pid_t pid1 = fork();
    if (pid1 == 0) {
        // 자식1: ls -l
        // [황금 틸] exec() 전에 stdout을 파이프 쓰기 끝으로 교체
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        execlp("ls", "ls", "-l", NULL);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) {
        // 자식2: grep .c
        // [황금 틸] stdin을 파이프 읽기 끝으로 교체
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[1]); // ← 이 줄을 빠뜨리면 grep은 영원히 기다립니다
        close(pipefd[0]);
        execlp("grep", "grep", "\\..c$", NULL);
    }
}
```

```
// 부모: 양 끝 닫기 (자식들이 EOF를 받을 수 있도록)
close(pipefd[0]);
close(pipefd[1]);
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
return 0;
}
```

이미지 — 원본 이미지

핵심 함정: 부모가 `pipefd[1]` (쓰기 끝)을 닫지 않으면, `grep` 은 파이프의 모든 쓰기 끝이 닫힐 때까지 EOF를 받지 못합니다. 프로세스가 영원히 블로킹되는 미스터리의 원인 중 하나입니다.

그리고 이 패턴의 응용은 Docker 컨테이너까지 이어집니다. Docker 실행의 실체는 `fork()` 후 `exec()` 전의 황금 틈에서 PID/네트워크/마운트 네임스페이스를 분리하고, 루트 파일시스템을 교체하는 것입니다.

```
1 # 직접 네임스페이스 격리 체험
2 sudo unshare --pid --mount-proc --fork /bin/bash
3 # 새 셸에서 ps aux -> PID 1부터 시작하는 완전히 격리된 세계
```

이미지 — 원본 이미지

3부. 메모리 페이지 교체 — 완벽한 이론을 포기한 실용의 아름다움

1. 왜 OS는 메모리를 '내보내고 불러오는가'

책상 위에 올려놓을 수 있는 서류의 양은 한정되어 있습니다. 그래서 우리는 당장 필요 없는 서류를 서랍에 넣고, 필요할 때 꺼냅니다. OS의 가상 메모리 시스템이 하는 일이 정확히 이것입니다.

- **수요 페이징(Demand Paging):** 프로그램이 실제로 접근할 때 비로소 물리 RAM에 페이지를 올립니다.

- **페이지 폴트(Page Fault):** 접근하려는 페이지가 RAM에 없을 때 발생하는 인터럽트. 커널이 디스크(스왑)에서 페이지를 읽어옵니다.
- **페이지 교체(Page Replacement):** RAM이 가득 찼을 때, 새 페이지를 들여오려면 기존 페이지를 스왑으로 내보내야 합니다. 어떤 페이지를 내보낼 것인가 — 이것이 알고리즘의 핵심입니다.

잘못된 선택은 **스래싱(Thrashing)** 을 낳습니다. 실제 작업보다 페이지를 교체하는 데 더 많은 시간을 소비하는, 시스템이 경련을 일으키는 상태입니다.

핵심 딜레마: 앞으로 가장 오래 쓰이지 않을 페이지를 내보내면 최적이지만, 미래는 알 수 없습니다.

2. OPT에서 Linux까지 — 이상과 현실의 타협사

Bélády의 OPT: 이론적 완벽함

미래 참조 패턴을 알 수 있다면 '가장 나중에 사용될 페이지'를 교체하면 됩니다. 이것이 Bélády의 최적(OPT) 알고리즘입니다. 성능은 최고지만 실제로 구현할 수 없습니다. 미래를 알아야 하기 때문이죠. 다만 다른 알고리즘의 성능 기준(벤치마크)으로 활용됩니다.

참조 열: 1, 2, 3, 4, 2, 3, 1, 5 / 프레임 수: 3

시점 참조 프레임 상태 폴트?

4 4 [4, 2, 3] ✓ ← 1을 교체 (다음 참조는 시점 7로 가장 먼 미래)

7 1 [1, 2, 3] ✓ ← 4를 교체 (이후 참조 없음)

LRU: 과거로 미래를 예측하다

"최근에 쓰지 않은 것은 앞으로도 쓰지 않을 것이다"는 직관으로 동작합니다. OPT에 근접한 성능을 보이지만, 문제가 있습니다. 모든 메모리 접근마다 타임스탬프를 갱신해야 하는 **하드웨어 비용이 막대**합니다. 대용량 메모리 환경에서는 오버헤드가 폭발적으로 증가합니다.

NRU / Clock: 실용적 근사의 아름다움

Linux의 핵심 아이디어는 R-bit(참조 비트) 하나로 LRU를 근사하는 것입니다.

페이지 우선순위 (낮을수록 먼저 교체):

$R=0$, $D=0$ 클래스 0: 최근 미사용 & 미수정 → 최우선 교체 대상

R=0, D=1 클래스 1: 최근 미사용 & 수정됨 (스왑 기록 필요)

R=1, D=0 클래스 2: 최근 사용 & 미수정

R=1, D=1 클래스 3: 최근 사용 & 수정됨 → 최후순위

시계 바늘이 원형 버퍼를 돌며 R-bit가 1이면 0으로 초기화("기회를 한 번 더 줍니다")하고, 0이면 교체 대상으로 선정합니다. 이론의 완벽함을 포기하고 단 두 비트로 실용적인 근사를 달성한 것 — 이것이 엔지니어링의 미학입니다.

Linux의 실제 구현: Active / Inactive 리스트

현대 Linux 커널은 단순 Clock을 넘어 **두 개의 LRU 리스트**로 정교하게 관리합니다.

[Active 리스트] ↔ [Inactive 리스트]

자주 참조됨 참조 빈도 낮음

(보호 대상) (교체 후보)

흐름:

새 페이지 → Inactive 리스트에 추가

재참조 시 → Active 리스트로 '승격'(Promotion)

Active 과부하 → 오래된 페이지가 Inactive로 '강등'(Demotion)

Inactive 부족 → 스왑 아웃 또는 해제

이 구조는 `/proc/meminfo` 에서 직접 눈으로 확인할 수 있습니다.

Active(anon): 890123 kB ← 스택·힙 중 활성 (스왑 보호)

Inactive(anon): 234567 kB ← 스왑 1순위 후보

Active(file): 344444 kB ← 파일 캐시 중 활성

Inactive(file): 333323 kB ← 메모리 압박 시 드롭 대상

아래 시뮬레이션에서 참조열과 프레임 수를 바꿔보면, 페이지 교체 알고리즘의 차이를 직관적으로 확인할 수 있습니다.

3. 시스템의 숨결을 읽는 법 — 메모리 압박의 현장

```
1 # 스왑 활동 실시간 관찰 (si=스왑 인, so=스왑 아웃)
2 vmstat 1 10
3 # si/so가 지속적으로 0이 아니라면 메모리 압박이 진행 중입니다
4
5 # 프로세스별 페이지 폴트 분석
6 ps -o pid,minflt,majflt,comm -p $(pgrep nginx | head -1)
7 # minflt(마이너): 디스크 없이 처리 / majflt(메이저): 디스크에서 읽어야 함
8
9 # TLB miss와 페이지 폴트 측정
10 perf stat -e page-faults,dTLB-load-misses ./my_program
```

4. 메모리를 내 뜻대로 — mlock, OOM, Huge Page

민감 데이터를 RAM에 고정하기:

Redis, 실시간 금융 시스템, 암호화 키 저장소는 `mlock()` 으로 데이터가 스왑 파일에 기록되는 것을 방지합니다. 스왑 파일은 디스크에 남는 평문의 흔적이기 때문입니다.

```
// 중요 데이터를 RAM에서 절대 내보내지 않도록 잠금
void *ptr = mmap(NULL, size, PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
mlock(ptr, size); // 이제 이 영역은 스왑으로 나가지 않습니다
// ... 암호키 또는 민감 데이터 처리 ...
munlock(ptr, size);
munmap(ptr, size);
```

OOM Killer 제어:

```
1 # 데이터베이스 프로세스를 OOM Killer로부터 보호
2 echo -1000 | sudo tee /proc/$(pgrep postgres)/oom_score_adj
3 # -1000: 절대 죽이지 않음 / +1000: 가장 먼저 죽음
```

Huge Page로 TLB 압박 완화:

```
1 # 2MB 페이지 100개 예약 (= 200MB, TLB miss 극적 감소)
2 echo 100 | sudo tee /proc/sys/vm/nr_hugepages
3
4 # SSD 환경에서 불필요한 스와핑 억제
5 sudo sysctl vm.swappiness=10
```

4부. I/O의 진화 — 기다림을 없애는 여정

1. 왜 I/O는 병목이 되는가

CPU는 나노초(ns)로 생각하지만, 디스크와 네트워크는 마이크로초(μ s)에서 밀리초(ms)로 답합니다. 이 수천~수백만 배의 속도 차이가 I/O 병목의 근본 원인입니다.

초기 Unix는 단순 블로킹 I/O로 충분했습니다. 하나의 연결, 하나의 프로세스. 그런데 1999년 Dan Kegel이 "C10K 문제"를 제기합니다. "어떻게 하면 서버 하나가 10,000개의 동시 연결을 처리할 수 있는가?" 그 질문이 I/O 모델의 진화를 촉발했습니다.

[I/O 모델 진화 연대기]

1970s Blocking I/O → 단순하나 확장 불가
1983 select() BSD 4.2 → 다중 FD 감시, O(n) 스캔
1994 poll() POSIX → select 개선, 여전히 O(n)
2002 epoll Linux 2.5 → O(1) 이벤트, C10K 해결
2019 io_uring Linux 5.1 → 시스템 콜 자체를 제거

2. 블로킹에서 링 버퍼까지 — 네 가지 아키텍처

Blocking I/O: 기다리는 것

프로세스 ————— read(fd) —————> 커널
 [블로킹] (디스크/네트워크 I/O 완료 대기)
 <———— 데이터 반환 —————

직관적이지만, 10,000 연결 = 10,000 스레드가 필요합니다. 메모리와 컨텍스트 스위치 비용이 폭발합니다.

select() / poll(): 동시에 여러 문을 두드리다

`select()` 는 여러 FD를 한 번에 감시하다가 하나라도 준비되면 반환합니다. 문제는 준비된 FD를 찾으려면 전체를 다시 순회해야 한다는 것(O(N) 스캔), 매번 `fd_set`을 커널에 복사해야 한다는 것, 그리고 FD 수 한계(기본 1,024)입니다.

한계	내용
FD 수 제한	<code>FD_SETSIZE</code> = 1,024 (기본값)
O(N) 스캔	연결 수 증가 시 선형적으로 느려짐
매번 재전달	매 호출마다 <code>fd_set</code> 을 커널에 복사
결과 재스캔	어떤 fd가 준비됐는지 앱이 직접 순회

epoll: 준비된 것만 알려주는 안내자

```

epoll_fd = epoll_create1(0)           ← 커널에 감시 테이블 생성
epoll_ctl(epoll_fd, EPOLL_CTL_ADD, ...) ← O(log N), red-black 트리 등록
n = epoll_wait(epoll_fd, events, ...) ← 블로킹, 준비된 FD만 O(1) 반환
for (i = 0; i < n; i++) {
    handle(events[i].data.fd);        ← 준비된 것만 처리
}

```

커널이 감시 상태를 유지하고, 준비된 FD만 직접 반환합니다. C10K 문제를 해결한 바로 그 혁신입니다.

io_uring: 시스템 콜 자체를 없애다

[사용자 공간] [커널 공간]

SQ Ring (제출 큐) ←공유 메모리→ SQ Ring 처리

CQ Ring (완료 큐) ←공유 메모리→ CQ Ring 채우기

사용자가 SQ Ring에 요청을 직접 기록하고(시스템 콜 없음), 커널이 비동기로 처리한 뒤 CQ Ring에 결과를 기록합니다(시스템 콜 없음). SQPOLL 모드에서는 `io_uring_enter()` 조차 없어집니다. **완전한 제로-시스템콜 I/O**입니다.

모델	동시 연결	시스템 콜 비용	복잡도
Blocking I/O	수백	매우 높음	낮음
select/poll	1,024 / 무제한	높음 O(N)	중간
epoll	수백만	낮음 O(1)	높음
io_uring	수백만+	거의 없음	매우 높음

3. 시스템 콜의 흐름을 눈으로 보다

```

1 # Nginx가 실제로 epoll을 어떻게 호출하는지 추적
2 strace -p $(pgrep nginx | head -1) -e trace=epoll_wait,epoll_ctl,accept4 2>&1 | head
  -20
3
4 # TCP 소켓 상태 분포 - ESTABLISHED가 몇 개인지 확인

```

```
5 ss -tan | awk 'NR>1 {print $1}' | sort | uniq -c | sort -rn
6
7 # Recv-Q가 큰 연결 - 서버가 처리 속도를 못 따라가는 신호
8 ss -tn | awk '$3 > 0'
9
10 # TCP 재전송 통계
11 netstat -s | grep -E "retransmit|failed"
```

4. 직접 만드는 epoll 서버와 io_uring 예제

epoll 기반 에코 서버 (Edge-Triggered 패턴):

```
int epoll_fd = epoll_create1(0);
struct epoll_event ev = { .events = EPOLLIN, .data.fd = server_fd };
epoll_ctl(epoll_fd, EPOLL_CTL_ADD, server_fd, &ev);

while (1) {
    int n = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
    for (int i = 0; i < n; i++) {
        if (events[i].data.fd == server_fd) {
            // 새 연결 - ET 모드로 등록
            int cfd = accept4(server_fd, NULL, NULL, SOCK_NONBLOCK);
            struct epoll_event cev = {
                .events = EPOLLIN | EPOLLET, // Edge Triggered
                .data.fd = cfd
            };
            epoll_ctl(epoll_fd, EPOLL_CTL_ADD, cfd, &cev);
        } else {
            // 데이터 읽기 - 버퍼를 한 번에 완전히 비워야 합니다
            char buf[4096];
            ssize_t len = read(events[i].data.fd, buf, sizeof(buf));
            if (len <= 0) { close(events[i].data.fd); }
            else { write(events[i].data.fd, buf, len); }
        }
    }
}
```


ET 모드 주의: Edge-Triggered는 상태 변화 시 단 한 번만 알립니다. 버퍼를 완전히 비우지 않으면 다음 이벤트를 받지 못합니다. 반드시 논블로킹 FD와 함께 사용해야 합니다.

io_uring로 파일 읽기 (liburing):

```
struct io_uring ring;
io_uring_queue_init(1, &ring, 0);          // 커널과 링 버퍼 공유 시작

struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
io_uring_prep_read(sqe, fd, buf, 255, 0);  // 요청 등록 (syscall 없음)

io_uring_submit(&ring);                    // 단 하나의 syscall로 배치 제출

struct io_uring_cqe *cqe;
io_uring_wait_cqe(&ring, &cqe);           // 완료 대기
printf("읽은 바이트: %d\n", cqe->res);
io_uring_cqe_seen(&ring, cqe);            // CQE 소비 완료
```

고성능 서버 커널 파라미터 튜닝:

```
1 # 동시 연결 한계 제거
2 sudo sysctl -w net.core.somaxconn=65535
3 sudo sysctl -w fs.file-max=2097152
4
5 # TCP 버퍼 최적화 (128MB)
6 sudo sysctl -w net.core.rmem_max=134217728
7 sudo sysctl -w net.core.wmem_max=134217728
8
9 # TIME_WAIT 재사용 (단기 연결 많은 API 서버)
10 sudo sysctl -w net.ipv4.tcp_tw_reuse=1
11 sudo sysctl -w net.ipv4.tcp_fin_timeout=15
```

현실에서 이 선택들이 어떻게 갈리는지, 우리가 매일 쓰는 도구들을 보면 명확합니다.

서버/런타임	I/O 모델	특징
Redis	epoll (단일 스레드)	단일 이벤트 루프, 메모리 연산으로 지연 최소화
Nginx	epoll (멀티 워커)	워커당 epoll, REUSEPORT로 커널 레벨 분산
Node.js	libuv → epoll/io_uring	이벤트 루프 + 스레드 풀 하이브리드
Tokio(Rust)	epoll/io_uring + async	io_uring 백엔드 전환 진행 중
PostgreSQL	블로킹 + 프로세스 연결	전통적 1:1, pg_bouncer로 보완

마치며 — 타협의 산물

fork()/exec() 분리, 메모리 페이지 교체, I/O 모델의 진화. 이 세 이야기는 겉으로는 전혀 달라 보이지만, 하나의 같은 질문에서 파생된 해결책들입니다.

"완벽한 이론과 현실의 제약 사이에서, 어떻게 가장 우아한 타협을 이뤄낼 것인가?"